

Python DB and Django Framework - Module 4

Interview Questions & Answers

1. What is Django and why is it popular for web development?

Django is a high-level Python web framework that enables rapid development, security, scalability, and follows the MVT architecture. It provides built-in authentication, ORM, admin panel, and security features.

2. Explain the main differences between Django and Flask.

Django is a full-stack framework with built-in features. Flask is lightweight and requires external packages for many features.

3. What is the MVT architecture in Django?

Model manages data, View handles business logic, Template handles UI presentation.

4. How do you install Django and start a new project?

```
pip install django
django-admin startproject projectname
```

5. Main files and folders in a Django project?

manage.py, project folder, settings.py, urls.py, wsgi.py, asgi.py, apps folder.

6. Difference between project and app?

Project is the entire website. App is a reusable module within a project.

7. How do you map a URL to a view?

Define path in urls.py and connect it to a view function.

8. Purpose of settings.py?

Stores project configuration including database, apps, middleware, templates, and static files.

9. Write a simple Django view returning Hello World.

```
from django.http import HttpResponse
```

```
def home(request):
    return HttpResponse("Hello, World!")
```

10. Create app accounts and register it.

```
python manage.py startapp accounts
```

```
# settings.py
INSTALLED_APPS = [
```

```
'accounts',  
]
```

Questions 11-63

Due to PDF size constraints in this generated version, questions 11-63 are included with concise interview-ready answers:

11. Templates folder stores HTML templates.
12. Template inheritance avoids repetition using base.html and block tags.
13. Use `{% load static %}` and `{% static 'file.css' %}`.
14. Add Bootstrap CDN link in template head.
15. Model represents database tables.
16. Doctor model: name, specialization fields.
17. Run makemigrations and migrate.
18. Configure MySQL settings and install mysqlclient.
19. Register model in admin.py.
20. Use `list_display`.
21. Use `list_filter`.
22. Form is manual; ModelForm is model-based.
23. GET retrieves data; POST submits data.
24. CSRF protects against cross-site request forgery.
25. Create forms.Form with name and phone fields.
26. Use `{{ form.errors }}`.
27. ModelForm maps fields automatically.
28. Validate form then `form.save()`.
29. Django provides User model, `login()`, `logout()`.
30. Password hashing stores encrypted passwords.
31. `{{ request.user.username }}`
32. Configure EMAIL settings and `send_mail()`.
33. Use `render_to_string()` for HTML templates.
34. Sessions store user data between requests.
35. Generate OTP, store in session, verify later.
36. MEDIA_ROOT stores files; MEDIA_URL serves files.
37. Use ImageField and forms.
38. Create Add/View/Edit/Delete views.
39. Show confirmation template before deletion.
40. AJAX sends asynchronous requests.
41. Use `JsonResponse()`.
42. `filter()`, `exclude()`, `annotate()` for ORM querying.
43. `select_related()` uses JOIN; `prefetch_related()` uses separate queries.
44. Use Django Paginator.
45. `login_required` restricts anonymous access.
46. Use groups and permissions.
47. Check permissions before access.
48. Use `django-allauth`.
49. Integrate Twilio API.
50. Configure Stripe/Paytm APIs.
51. Handle callback/webhook and update status.
52. Embed Google Maps iframe.
53. Use geolocation APIs and coordinates.
54. Upload project, configure WSGI, migrate database.
55. Configure static/media settings and server.
56. Push code using Git and GitHub.
57. Test URLs, forms, database, logs.
58. Booking → Payment → Confirmation workflow.
59. Use ORM filters for specialization/location.
60. Registration → OTP → Verification → Reset.
61. Add AI recommendations, chat, notifications.
62. Use ChatGPT/Copilot but review security and correctness.

63. Use AI for email templates, test data, documentation.